Adapting IDS with few newly labelled attacks

Intrusion detection system (IDS) · Classify network flows as benign or attack types

An attack absent from the initial labelled data may appear later
A fixed test score alone does not show how well a detector can adapt

At deployment

Benign + attacks A and B
Labels for a limited set of types

After investigation

A few labelled cases of C
New labels for the detector

After updating

Detection of C and update cost
Quality on earlier classes

XGBoost Refit the model on all available labels, including the new cases

TabPFN Supply existing and new cases as context to the pretrained model

Evaluation scope · Adaptation after labels for a new attack become available

TabPFN · Pretrained weights and supplied examples

Prediction rules pretrained on synthetic tabular tasks · TabPFN-3, arXiv 2026

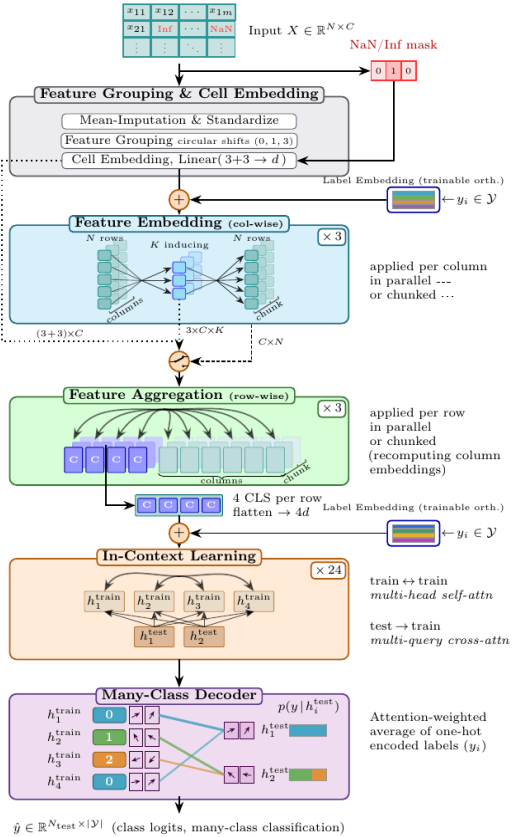
Pretraining learns how to predict from examples
Examples defining the current IDS classes are supplied at prediction time



In-context learning Prediction conditioned on the supplied context; weights unchanged

Fine-tuning Split IDS examples into context/query and backpropagate query loss
Additional weight training; not performed in this comparison

v3 architecture · Representing network flows



1 Encode values · Cell embedding

Encode each value, such as packet length
Include missing-value indicators

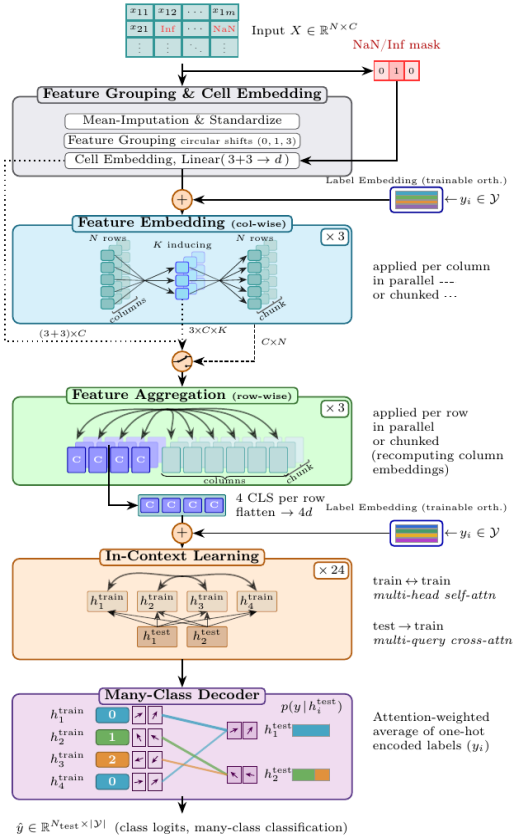
2 Summarize columns · Feature embedding

Summarize each feature across flows
Use the labels in the context

3 Combine features · Feature aggregation

Combine a flow's features into a representation
Prepare it for comparison with other flows

v3 architecture · Context and update computation



4 Consult examples · In-context learning

Compare the query with context examples
Use attention to weight their relevance

5 Predict probabilities · Many-class decoder

Weight and combine the context labels
Retain benign, existing and new attack examples

Update computation without weight training

Context preprocessing · Representations · Cache
Measure the cost against XGB refitting

Evaluation data · From benchmark rows to labelled examples

One row = features of a recorded network flow + its benign/attack label
Clean and split each benchmark separately; select examples within that benchmark

CIC2018	**Train · Candidate examples** Select initial cases, then new attacks	**Test · Fixed evaluation flows** 3,042,473 rows · Excluded from updates
ToN-IoT	**Train · Candidate examples** Select initial cases, then new attacks	**Test · Fixed evaluation flows** 2,271,723 rows · Excluded from updates

CIC example · Initial 1,000 + 100 DDoS rows from the same CIC train = 1,100 labels

Use these same 1,100 rows for XGB refitting or as PFN context
“Current dataset” means the labelled examples supplied so far

Evaluation scenario · n labels per new attack class

Counts = newly supplied training/context examples at each stage · Ordered by class arrival

CIC2018

Stage	Newly supplied class	Per class	Total so far
1	Benign Brute force DoS	750 125 125	1,000
2	DDoS	n	1,000 + n
3	Web attacks	n	1,000 + 2n
4	Infiltration	n	1,000 + 3n
5	Bot	n	1,000 + 4n

ToN-IoT

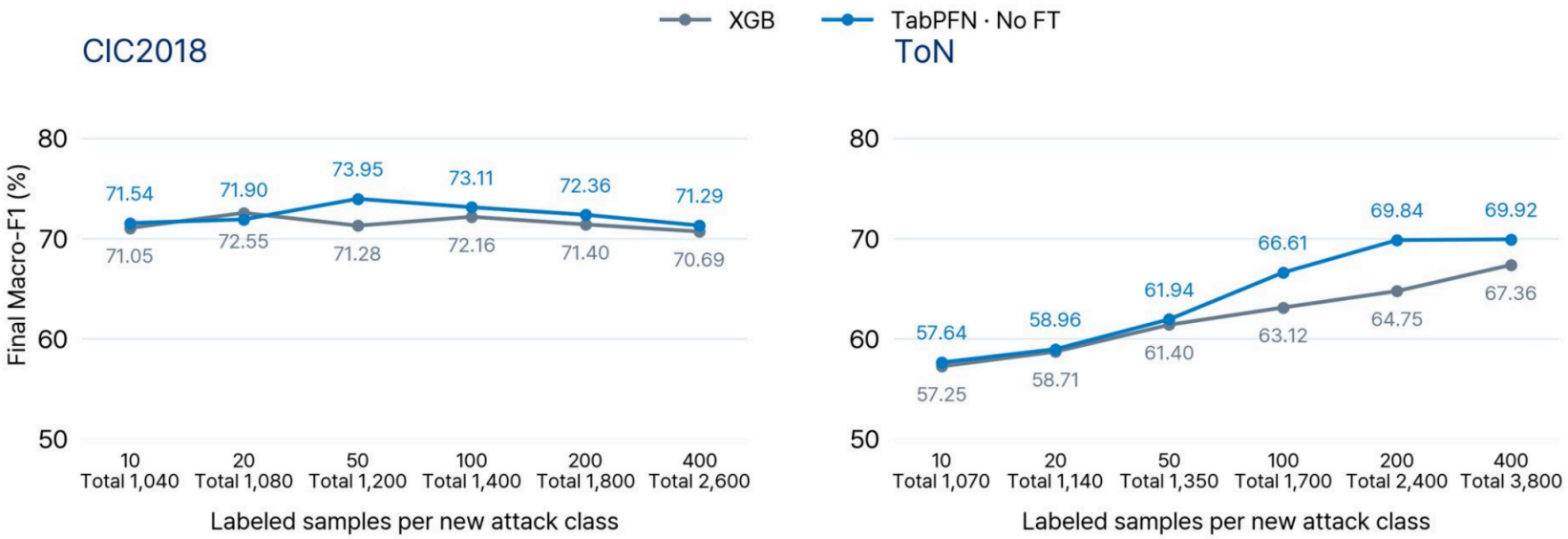
Stage	Newly supplied class	Per class	Total so far
1	Benign DoS Scanning	750 125 125	1,000
2	DDoS Injection	n n	1,000 + 2n
3	Password	n	1,000 + 3n
4	XSS	n	1,000 + 4n
5	Backdoor Ransomware	n n	1,000 + 6n
6	MitM	n	1,000 + 7n

At n = 100: CIC 1,400 / ToN 1,700 total labels; n is a label budget, not daily traffic volume

Completed sweep: n = 10, 20, 50, 100, 200, 400; 100 is illustrative, not an optimum

Completed cumulative study · Quality versus labels

Initial 1,000 · 10 to 400 labels/new class · Cumulative data · Final model on full test

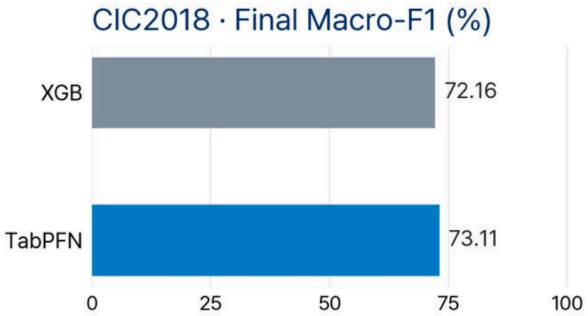


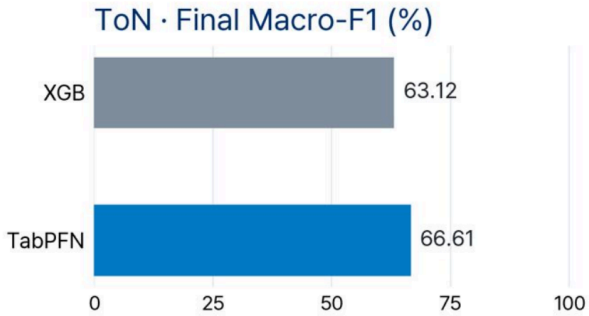
Overall quality as the number of available labels changes
New-attack detection and before/after retention require separate measurements

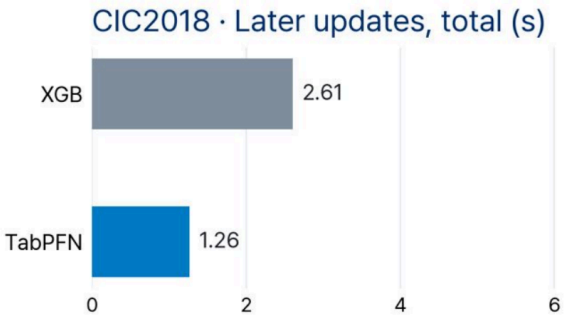
Completed cumulative experiment · Quality and update cost

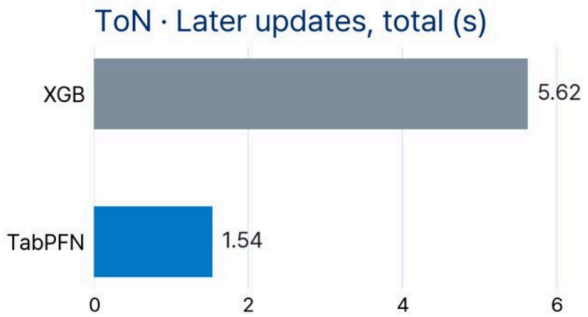
Illustrative n = 100 · XGB cumulative refit / PFN cumulative context · No IDS fine-tuning

Update cost = sum over stages after stage 1; initial preparation and inference excluded









Measured costs for this small cumulative setting; fixed-buffer retention remains to be tested